Plexus

A blueprint for differentiable tissue: hierarchical sets, operators, and a formal simulation language

Janelia Research Campus

July 6, 2026

Abstract

Living tissues couple biochemical signalling, cellular interaction, mechanical force, and collective dynamics across scales. The frameworks built so far (CellGraph, NeuralGraph, MPMGraph, MetabolismGraph) each implement *one* graph type; a tissue needs them coexisting. This document is a single blueprint, distilled from a working prototype, for building one repository that unifies them. It has three parts: (i) the missing primitive — a **hierarchical graph container** stated in the language of **sets** and **operators**; (ii) a **formal intermediate representation** — a typed, declarative spec language compiled to a calculus of registered operators — that turns intent into a runnable simulation and that a person or a language model can drive; and (iii) the **experiment** — ten distinct simulations and an emergent ant-trail model, all produced from the same code by changing only the simulation file — with the lessons that follow for how to construct the repository.

Part I — The abstraction

1 Two entities: sets and fields

The container is built from two kinds of object: **sets** and **fields**. A **set** S_k is a discrete collection of like elements at one scale — particles, cells, populations, organisms, or the compartments inside a cell: membrane particles, cytoplasm, nuclei, and metabolites (Fig. 1). Every element of a set shares the same state schema and the same operators, and a set is stored as batched tensors — never one object per element.

Sets are linked by **containment**. A parent map

$$\pi : S_k \longrightarrow S_{k+1}$$

sends each element to the parent that contains it — a particle to its cell, a cell to its population. A cell is therefore not a single object but the collection of everything that maps to it: its particles, nucleus, metabolites, and any other child sets. Stacked across scales, these maps form the *containment graph* of the next section.

A **field** $f : \Omega \to \mathbb{R}^c$ is a continuous quantity defined on its own grid — morphogen concentration, pressure, substrate stiffness, or an extracellular chemical. Fields are *not* part of the containment graph: they do not contain cells, and cells do not contain fields. Instead, each field is coupled to

one or more sets through *exchange* operators — secretion, sensing, particle–grid transfer, or field sampling (Section 3).

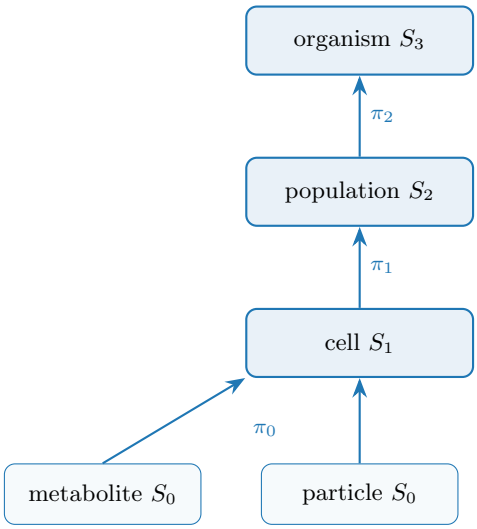


Figure 1: Containment across scales: each parent map π sends an element to the one that contains it. A cell is the collection of everything that maps to it — here its particles and metabolites; in general also its membrane, nucleus, and other child sets (Section 2).

2 Containment: a graph of sets

Section 1 named the sets and joined them with parent maps. Two points make the container precise, and they are what keep the engine free of branching.

One set per kind of object. Rather than a single set whose elements carry a label saying what each one is — which forces every operator to ask "what am I looking at?" at run time — we keep *one set per kind*: membrane particles, nuclei, metabolites, and cells are separate sets, each with its own state schema, operators, and fields. An operator then names the one set it acts on (`at: nucleus`) and never branches. This is the same discipline an entity–component system or a relational database already uses.

Joined by containment maps. The sets are joined by parent maps, declared simply by naming the parent,

$$\pi : S_k \longrightarrow S_{k+1},$$

which send every element to the parent that contains it. A parent may have many children, so containment is a **graph**, not a chain: one cell gathers several child sets at once (Fig. 2). The engine needs only two things — `Level.parent` (which parent each element has) and `Level.parent_name` (which set that parent belongs to) — so new biological structure is just a new set and a parent map, never an engine change.

A new set, or a new type? The rule (Table 1): two groups that differ in *state or operators* are *different sets*; two groups that share both and differ only in *parameter values* are *types within one set* — the `types:` block. A cell and a nucleus have different state and dynamics, so they are

different sets; a soft cell and a stiff cell differ only in stiffness, so they are two types of the single cell set.

	Different sets	**Types within one set**
example	membrane vs. nucleus vs. metabolite	soft cell vs. stiff cell
differ in	state space + operators	only parameter values
in the spec	separate `sets:` with `parent:`	a `types:` block inside one set
dispatch	by name (`at: nucleus`)	per-element parameter lookup

Table 1: When to declare a new set versus a new type: a different *kind* (state space or dynamics) ⇒ a new set; a parametric *variant* ⇒ a type within a set.

Keeping each operator attached to its own set is the payoff: `membrane_tension` acts on `membrane_particle`, `metabolism` on `metabolite`, and `gene_regulation` on `nucleus`, while Exchange operators connect compartments explicitly (nucleus ↔ cytoplasm, membrane ↔ extracellular field) — never one mega-operator branching on a role flag.

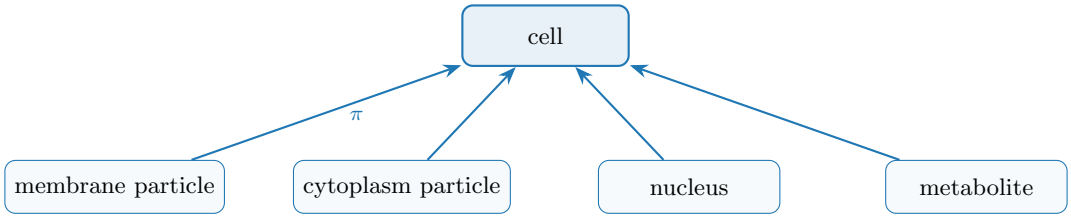


Figure 2: A containment graph: one parent set (`cell`) with several child sets, each with its own parent map. The cell is the collection of these children — not a bag of one kind of particle. A new compartment is a new row in the spec (Listing 1), not an engine edit.

Listing 1: A cell as a collection of child sets: each compartment is its own set with a parent map.

```
sets:
  cell:               {n: 100}
  membrane_particle:  {parent: cell, per_parent: 64,  role: membrane}
  cytoplasm_particle: {parent: cell, per_parent: 200, role: cytoplasm}
  nucleus:            {parent: cell, per_parent: 1,   role: nucleus}
  metabolite:         {parent: cell, per_parent: 50,  role: metabolite}
```

3 Operators

A GNN is an **operator** $\mathcal{O}$ dispatched by the relation it acts on. An operator changes exactly one of the three things the container holds (Fig. 3): the **state** on the nodes, the **relations** between them, or the **entities** themselves. **Four dynamics operators** move *state* along a fixed structure, one per relation the container admits, each returning a time-derivative contribution. **Rewire** changes the *relations* — the edge set E, which nodes interact. **Divide** and **Die** change the *entities* — the node set $|S_k|$, which nodes exist. Those last three return no state delta. The four dynamics

operators:

Lateral	$\mathcal{O}_E,\ E \subseteq S_k \times S_k$	(ODE interaction + graph Laplacian)
Aggregate $\uparrow$	$\sum_\pi : S_k \to S_{k+1}$	(reduction over children of π)
Broadcast $\downarrow$	$\pi^* : S_{k+1} \to S_k$	(lift along π)
Exchange	$\mathcal{S}, \mathcal{G} : S_k \leftrightarrow F$	(scatter / gather; bipartite)

Two pieces of notation: $S_k \times S_k$ is the set of all node *pairs*, so an edge set $E \subseteq S_k \times S_k$ is simply a graph on S_k — which nodes interact (e.g. neighbours within a cutoff radius, or a fixed connectome). And $\mathcal{S}, \mathcal{G}$ are the two halves of a field coupling: *scatter* $\mathcal{S} : S_k \to F$ writes object state into the field, while *gather* $\mathcal{G} : F \to S_k$ reads it back. Each operator in turn:

Lateral $\mathcal{O}_E$. Message passing *within* one scale along the edges E. It carries the within-scale physics: pairwise interactions (forces, adhesion, signalling) plus a graph Laplacian that diffuses or smooths quantities over the edges. Examples: particle–particle elasticity, cell–cell flocking or adhesion, neuron–neuron signal propagation.

Aggregate $\sum_\pi$ (up). Pools each parent's children into one quantity — a cell's position is the mean of its particles, its metabolic state the sum over its molecules. This is how fine-scale dynamics are summarised into, and so move, the coarser object above.

Broadcast π^* (down). The adjoint of Aggregate: it copies a parent's state to each of its children, so a decision taken at the cell level (a body force, a polarity, a signalling output) is applied identically to all of that cell's particles. Aggregate and Broadcast are a matched up/down pair on the same map π.

Exchange $\mathcal{S}, \mathcal{G}$. Couples a set to a field: scatter $\mathcal{S}$ deposits/secretes object state into the field, gather $\mathcal{G}$ samples/senses the field back onto the objects. The relation is bipartite (objects $\leftrightarrow$ grid); it is exactly the MPM particle–grid transfer (P2G/G2P) and chemotactic secretion/sensing.

Rewire $\mathcal{R}$ (relations). Rewire changes the *relation* $E \subseteq S_k \times S_k$ on which the lateral operators act — *which* nodes are neighbours — and returns no state delta. A *geometric* relation is rebuilt every tick from the live configuration (a **radius graph**: all pairs within a cutoff; or a k-nearest-neighbour or Delaunay graph), so the interaction graph tracks motion, growth, and division; a *fixed* relation (a connectome) is built once and never rewired. Rewire is of a different nature from the other five: it neither moves state nor changes who exists — it maintains the *scaffold* the dynamics run on. Separating *who* interacts (Rewire) from *how* they interact (Lateral) is also what turns a dense $O(N^2)$ force law into $O(|E|)$ message passing and lets several lateral operators share one graph.

Divide and Die (entities). The dynamics operators move *state* and Rewire moves *relations*, both while the node set stays fixed; living matter also rewrites its own membership. **Divide** $\mathcal{B} : x \mapsto \{x', x''\}$ (proliferation) and **Die** $\mathcal{D} : x \mapsto \varnothing$ (apoptosis / efflux) are the only operators that change the *entities* themselves — the cardinality $|S_k|$. For a *living* system they are as primordial as the four dynamics operators: a tissue is defined as much by the cells it makes and loses as by how they move. Everything else assumes the cells it acts on already exist.

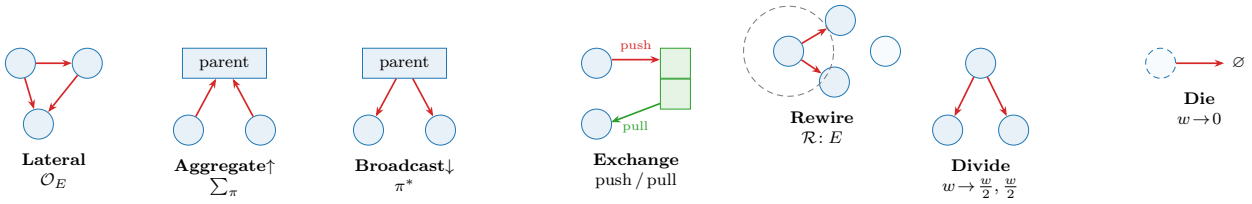


Figure 3: The operators, grouped by what they change. The first four are *dynamics*: they move *state* along a fixed structure — *Lateral* within a set; *Aggregate*/*Broadcast* across containment π; *Exchange* to/from a field (P2G/G2P and the metabolite–reaction graph are both Exchange). The last three return no state delta: *Rewire* changes the *relations* — the edge set E, which nodes interact (a radius graph rebuilt each tick, dashed cutoff); *Divide* and *Die* change the *entities* — the node set $|S_k|$, cell division and death on a fixed buffer with per-element occupancy w.

Differentiating division and death. Changing $|S_k|$ is the one move that resists automatic differentiation: it alters a tensor's shape, and "does this cell divide?" is a discrete event with no useful gradient. We keep the framework differentiable with a **continuous-occupancy** formulation. Rather than resize, we hold a fixed buffer of $N_{\max}$ slots and give every element a *mass* $w_i \in [0, 1]$: `Die` decays $w_i \to 0$ (the slot goes dormant, not deleted); `Divide` activates a dormant slot, copies the mother's state with a small perturbation, and splits the mass $w \to (\frac{w}{2}, \frac{w}{2})$. *Every operator then honours* w: Aggregate becomes a *mass-weighted* reduction $\big(\sum_i w_i x_i\big)/\sum_i w_i$, Lateral weights each message by w_j, and so on — so the boolean selector mask of Section 7 is just the $\{0, 1\}$ special case of w. The trajectory between events is now smooth in w, so gradients flow on a fixed-shape tensor (and `torch.compile` still applies). The one genuinely non-differentiable part — the integer decision of *when* to divide or die — is handled separately: a straight-through or Gumbel estimator to learn a division *policy*, a score-function (REINFORCE) gradient for an expected objective with respect to division/death *rates*, or black-box search (UCB/CEM) when the objective is itself discrete (final count, fraction escaped). This is the *use both* recipe of Part III applied to structure: gradients for the continuous interior, black-box for the discrete choices.

4 Dynamics: a schedule

A simulation is a multi-rate composition of operators, declared as a `Schedule`. In math terms, one tick is the composition $\Phi = \mathcal{O}_n \circ \cdots \circ \mathcal{O}_1$, and the dynamics is its iteration $x_{t+1} = \Phi(x_t)$ — a discrete flow. *Multi-rate* means the operators need not all fire on every tick: a slow process (diffusion, cell division) can be scheduled to act once every k ticks, while a fast one (mechanics) runs every tick, so a single `Schedule` interleaves dynamics that evolve on different time scales. The `Schedule` is just the code-level encoding of that composition. Each operator is *pure*: instead of editing the state in place, it only *returns its contribution* — a delta Δ_i (a time-derivative term) for the sets it touches. The schedule sums the deltas acting on each set and integrates once per tick.

An operator declares the *order* of its delta, since a time-derivative can be of position or of velocity. A *first-order* (overdamped) operator returns a velocity and integrates as $x_{t+1} = x_t + \Delta t \sum_i \Delta_i$; a *second-order* (inertial) operator returns an acceleration and integrates as $v_{t+1} = v_t + \Delta t \sum_i \Delta_i$, $x_{t+1} = x_t + \Delta t\, v_{t+1}$. The order is a fixed property of each operator (`EMIT`: `velocity` for first order, `acceleration` for second): attraction–repulsion is first order, boids flocking is second; all engine-integrated operators acting on one set must agree, so the set integrates as a single order. (A dynamics operator may instead declare `EMIT` = `mpm_acceleration` — an acceleration

consumed by the MPM substep as the external force a_{ext} rather than integrated by the engine.) Friction in the inertial case is itself an operator (a `drag` delta $-\kappa v$), not a special term in the integrator.

Two consequences follow: several operators may act on the same set in one tick (their deltas simply add — none overwrites another), and because nothing is mutated in place, gradients flow through the sums and the integrator, so the whole rollout stays differentiable. The LLM searches over schedules and operator parameterizations — one well-defined action space instead of per-domain code.

5 Glossary

Concept	Set/operator term	Symbol	Code
collection of like nodes	set (a level)	S_k	`Level`
a single node	element	$x \in S_k$	row of `Level.state`
learnable per-node code	embedding	a_i	`Level.embedding`
containment	parent map	$\pi_k : S_k \to S_{k+1}$	`.parent`, `.parent_name`
within-set links	relation	$E \subseteq S_k \times S_k$	`Level.edge_index`
continuum quantity	field	$f : \Omega \to \mathbb{R}^c$	`Field`
dynamics (GNN)	operator: ODE+Laplacian	$\mathcal{O}$	`Operator`
within-set op	lateral	$\mathcal{O}_E$	`Lateral`
up op	reduction over children	$\sum_\pi$	`Aggregate`
down op	lift	π^*	`Broadcast`
set↔field op	transfer	$\mathcal{S}, \mathcal{G}$	`Exchange`
rebuild the relation	relations: rewire	$\mathcal{R} : \to E$	`Rewire`
make an element	entities: division	$\mathcal{B} : x \mapsto \{x', x''\}$	`Divide`
remove an element	entities: death	$\mathcal{D} : x \mapsto \varnothing$	`Die`
per-element occupancy	occupancy	$w_i \in [0, 1]$	`Level.occ`
the container	hierarchy	$(S_\bullet, F_\bullet, \pi_\bullet)$	`Hierarchy`
the model	composition of ops	$\mathcal{O}_n \circ \cdots \circ \mathcal{O}_1$	`Schedule`

Part II — A formal intermediate representation

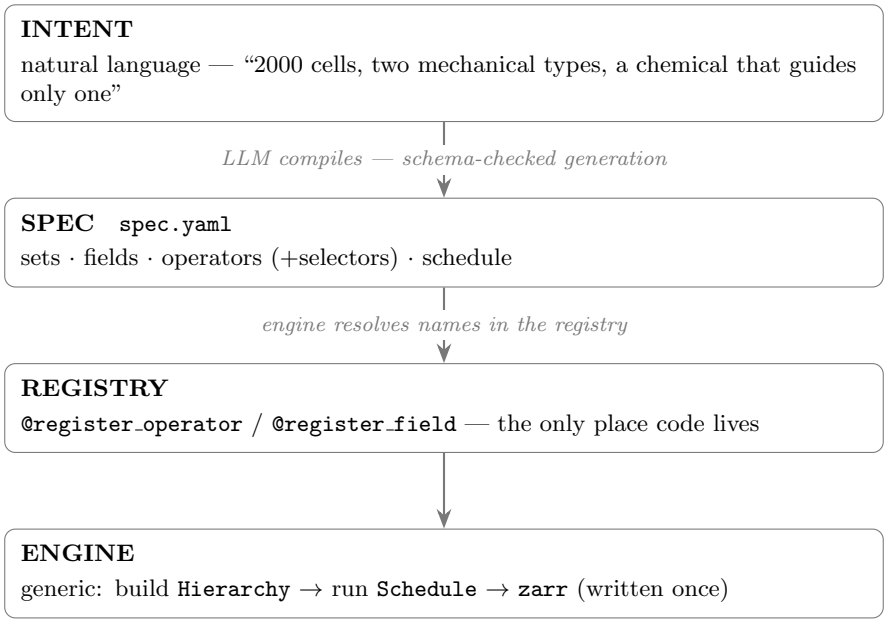
6 Two layers and the contract between them

The contribution of this part is not that a language model can drive the framework — it is that there is a **formal intermediate representation** for anything to drive at all. Between a person's *intent* and a running *simulation* we interpose one typed, declarative artifact, the **spec**, and the

system becomes a compiler stack:

$$\text{Intent} \longrightarrow \text{Language} \longrightarrow \text{Calculus} \longrightarrow \text{Simulation}.$$

Intent compiles to the **language** (the spec, `spec.yaml`); the spec names primitives in a **calculus** of operators (the registry); the engine **interprets** that calculus into a simulation. In one line: the spec is the *source*, the registered operators are the *primitives*, and the engine is the *interpreter*. A language model is one front end on the first arrow — useful, but replaceable; the durable object is the representation in the middle. Every property claimed below is a property of *it*, not of the model that writes it.



The **spec is the contract**. The LLM owns everything above it (intent → spec); the code owns everything below (operators). They never touch directly. This makes the system both LLM-drivable and human-auditable: the spec is small, typed, and reviewable, unlike the 500-line flat `config.py` it replaces.

Being a small, typed, declarative file gives the contract the properties one wants of an interface between an unreliable generator and exacting code:

- **Verifiable.** The spec is validated against the schema *before* anything runs: an unregistered operator, a missing required property, or a malformed selector is rejected up front, so a spec that loads is guaranteed runnable (the missing-operator and missing-property cases below). Beyond well-formedness, each simulation also ships an *intent* check that measures whether the requested behaviour actually occurred — "it ran" is not "it worked".
- **Reproducible.** A run is bit-deterministic given (spec, seed) — the engine forces deterministic kernels — so the same spec yields the same trajectory, and a fresh seed draws a new realization of the same dynamics.
- **Saveable.** The spec is ∼20 lines, so it version-controls, diffs, and archives trivially; together with the seed it *is* the complete record. We archive the recipe, not the dish: store spec + seed (plus metrics and a thumbnail) and regenerate the trajectory on demand (`archive.py`, `GALLERY.md`).

- **Composable.** A variant is a base spec plus a handful of overrides, which makes ablations and the optimizer's parameter sweeps cheap.
- **Extensible.** New behaviour is one new registered operator; the operator vocabulary grows monotonically, and every earlier spec keeps working.

7 The simulation language

The contract above is, in effect, a small **domain-specific language** in declarative-data form. Like any language it has *words* (vocabulary), *grammar* (how they combine — here a typed schema, which *is* the language definition: required keys, type fractions summing to one, every operator registered, every schedule token resolving), and *semantics* (what running them means: the engine builds the `Hierarchy` and iterates the schedule, $x_{t+1} = \Phi(x_t)$). Table 2 is the complete vocabulary; every construct in it appears in the example of Listing 2.

Structurally this is an **entity–component–system** (ECS) architecture: sets are entities, their state buffers are components, operators are systems, and the schedule is the ECS system pipeline. We adopt that framing deliberately — it is the cleanest mental model and matches the engine one-to-one. Each entity kind declares its component layout once in the registry (`@register_entity`): a typed *state schema* naming which state columns are position, velocity, and so on, plus *render* hints (what to colour by, whether to draw orientation). The engine, the operators, and the visualizer all read column semantics from this one declaration rather than hard-coding offsets like `state[:, :2]` — so a new entity is drawn and integrated correctly for free.

Most of the spec is plain typed key–value; the only genuine sub-grammars are the last two blocks of Table 2 — *selectors* and the *schedule*. Both are deliberately minimal but extend naturally: selectors to conjunctions and to geometric or temporal predicates (`cell[type=soft & loaded=0]`, `cell[x<0.5]`, `cell[age>10]`), and the schedule to explicit per-operator rates and nested loops.

We keep the language as *declarative data* (YAML/JSON) rather than a bespoke syntax, so an LLM can emit it under schema-constrained decoding and a human can diff, review, and archive it; the grammar is pinned by a typed schema (Pydantic or JSON-Schema), and composition, overrides, and parameter sweeps (variants, ablations, the optimizer) are delegated to a config layer such as Hydra rather than reinvented. Prior art worth borrowing: ECS engines (the scheduling model), SBML/Antimony (reaction–species vocabulary), NeuroML and MorpheusML/PhysiCell (multicellular specs), UFL (PDE operators), and NetLogo/Mesa (agent rules).

A complete spec is a single `spec.yaml`. Listing 2 assembles the vocabulary of Table 2 into one: two cell `types`, `particles` contained in each `cell`, one chemical `field`, and a `schedule`. Soft cells `deposit` the chemical and `chemotax` (climb) it, so they self-aggregate, while stiff cells — selected out by `cell[type=soft]` — do not.

Listing 2: A complete simulation: soft cells secrete and follow a chemical and self-aggregate.

```
general: {name: aggregate, seed: 0, n_frames: 250, dt: 0.05, boundary: periodic}
sets:
  cell:
    n: 100
    types: {soft: {fraction: 0.5, youngs: 12}, stiff: {fraction: 0.5, youngs: 300}}
  particle: {parent: cell, per_parent: 100, radius: 0.02}   # containment
fields:
  chemical: {frame: grid, res: 96, couples_to: cell}
operators:
  - {op: glide, at: cell, speed: 15.0, rot: 0.4}
```

```
  - {op: mls_mpm_mechanics, at: particle, n_grid: 128, substeps: 8, a_max: 800, drag: 4}
  - {op: deposit,  at: "cell[type=soft]", to: chemical, rate: 1.0}    # only pop 1 makes it
  - {op: diffuse,  at: chemical, rate: 0.1}                           # field self-dynamics (was chemical.
    diffuse)
  - {op: decay,    at: chemical, rate: 0.05}
  - {op: chemotax, at: "cell[type=soft]", from: chemical, gain: 150.0} # only pop 1 climbs it
schedule: [aggregate, [glide], deposit, diffuse, decay, chemotax, mls_mpm_mechanics]
```

The validator (prototype: `scenario_schema.py`) guarantees that a file which loads is runnable: every `op` exists; every `at`/`to`/`from` references a declared set/field; type `fraction`s sum to one; every schedule token resolves. (One trap: never use a YAML 1.1 boolean key such as `on`/`off` — we use `at`.)

8 Operator catalog

Operators are the system's vocabulary; it grows monotonically and every past spec keeps working. The catalog, grouped by what each operator changes (Section 3). `EMIT` is the temporal-integration state a dynamics operator's returned delta represents — `velocity` / `acceleration` integrated by the engine, `mpm_acceleration` routed to the MPM substep, or `---` for an operator that writes a field, a relation, or a `heading` and returns no set delta:

op	kind	EMIT	behaviour
Dynamics — move a set's state (return a delta)			
`attraction_repulsion`	lateral	velocity	per-type difference-of-Gaussians interaction law
`squared_law`	lateral	acceleration	inverse-square law: Coulomb electrostatics *or* Newtonian gravity (`law: charge/mass`)
`cohesion`, `alignment`, `separation`	lateral	acceleration	the three boids rules; their deltas sum to flocking
`cruise`	lateral	acceleration	Vicsek self-propulsion toward cruising speed v_0
`drag`	lateral	accel. | mpm_accel.	viscous friction $-\kappa v$ (`emit:` routes to the engine or the MPM substep)
`glide`	lateral	velocity	active self-propulsion along the `heading`
`gravity`, `mpm_anchor`, `mpm_spin`	lateral	mpm_accel.	uniform / rest-anchor / solid-spin body forces (MPM substep)
`bounce`	lateral	—	specular wall/obstacle re-heading (writes `heading`)
`chemotax`	exchange	vel. | mpm_accel.	climb/flee a field's gradient (`emit:` routes)
`sense`	exchange	—	Physarum 3-sensor pheromone steering (writes `heading`)
`deposit`	exchange	—	secrete/deposit set state into a field
`heading_align`, `flow_align`	exchange	—	align `heading` to neighbours / to solved MPM flow
`active_force`	exchange	mpm_accel.	activation-gradient body force (cardiac contraction)
`active_stress`	exchange	—	activation → active-stress tensor (`H.active_stress`)
`apply_material_map`	exchange	—	image field → per-particle material (`youngs`)
`agent_to_mpm`, `mpm_to_agent`, `agent_remodel`	exchange	— / vel.	two-way agent ↔ material coupling
`aggregate`	aggregate	—	parent state = mass-weighted mean of children (↑)
`broadcast`	broadcast	velocity	lift parent state onto its children (↓)
`mpm_strain`, `p2g`, `mpm_grid_update`, `g2p`	lat./exch./field	—	the decomposed MLS-MPM cycle (strain → P2G → grid solve → G2P)
`mls_mpm_mechanics`	exchange	—	the same cycle as one fenced *transitional* oracle (Section F)
Fields — a field's own dynamics (`kind=field`; write the grid, return {})			
`diffuse`, `decay`	field	—	Laplacian diffusion / linear evaporation
`pacemaker`	field	—	periodic scalar clock $p(t)$ → `H.signals`
`activation_pulse`	field	—	clocked activation field: shared-clock × profile, or per-pixel delayed wave
`playback`	field	—	advance a `prescribed` (video) field one frame
Relations — change the edge set E (`kind=rewire`)			
`radius_graph`	rewire	—	neighbour graph: live pairs within a cutoff, rebuilt each tick
Entities — change the node set |S| (`kind=structural`)			
`cell_divide`	structural	—	proliferation: wake a dormant buffer slot (`occ`) beside the mother

9 Natural language → simulation: the repeatable mapping

To make the LLM's compilation *repeatable* (same request → same structure), it follows fixed rules; the most useful:

1. "N cells/agents" → a set with `n: N`.
2. "made of K X per cell" → a contained set with `parent/per_parent` (two levels).
3. "two types differing by P" → `types:` with P as a per-type property.
4. "mechanical/elastic/rigid" → particles + the MPM operators; map stiffness to `youngs`.
5. "moves by boids / flock" → `cohesion+alignment+separation`; "wanders / motile" → `glide`.
6. "creates/secretes a chemical" → a `field` + `deposit`; "follows a gradient / chemotaxis" → `chemotax`; "ant / pheromone trails" → `sense` (Physarum 3-sensor).
7. "particles attract/repel, interact by type" → a `radius_graph` (the relation) followed by a lateral interaction op (e.g. `attraction_repulsion`); the per-type law parameters go in `types:`, read by the operator.
8. **"only population X does Y"** → the selector `at: cell[type=X]`. This is the highest-leverage primitive: it scopes behaviour to a subpopulation and lets two behaviours coexist.

Throughout, the mapping respects the set/field/operator split of Section 2: a *behaviour* becomes an operator, a *kind of node* a set (or a type within one), the *space* the `general:` block — never the reverse.

10 A spread of simulations from one registry

Once the vocabulary existed, qualitatively different behaviours were *just different specs* over the same code — breadth comes from the intermediate representation, not from new code. Each simulation is 100–600 cells of ∼100 MLS-MPM particles (the two cell types in two colours over the shaded chemical field); eight are shown in Fig. 4:

- **aggregate** — soft cells secrete a chemical and climb it, clumping together;
- **disperse** — soft cells flee their own chemical (negative gain), spreading apart;
- **swarm** — strong boids make the cells flock and move as one;
- **wander** — active motility alone: a diffusive gas of cells;
- **sort** — type-specific adhesion segregates the two populations;
- **chase** — a fast-decaying field makes moving hotspots the cells chase;
- **crystal** — pure repulsion spaces the cells into a lattice;
- **collapse** — low diffusion with strong chemotaxis pulls soft cells into one tight cluster.

Across all eight, only the operator choices and their parameters differ; the engine, the operators, and the schedule grammar are identical.

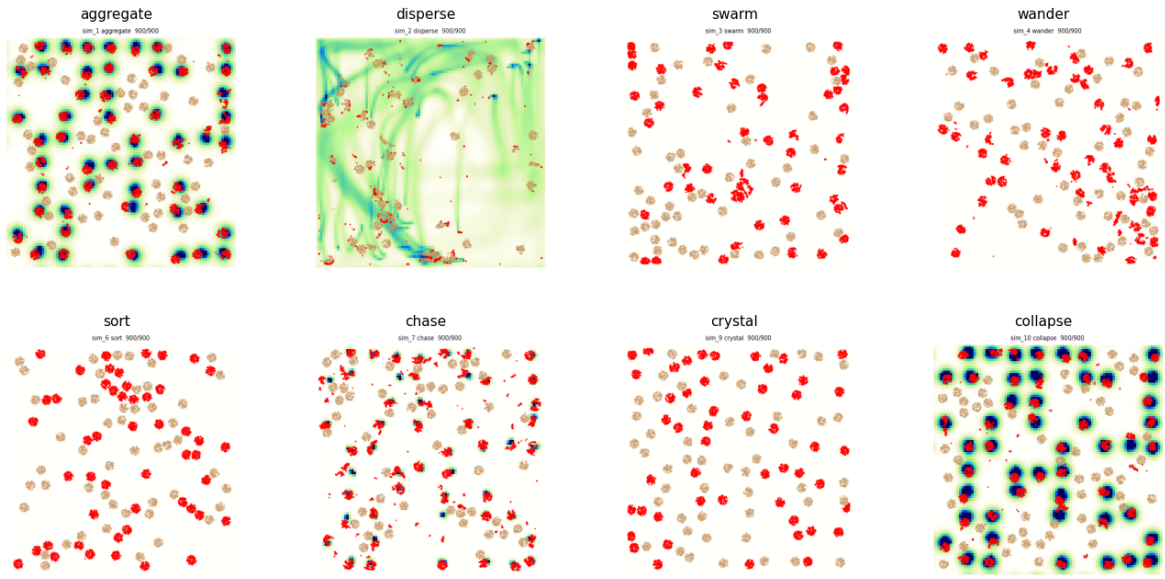


Figure 4: End states of the eight simulations described above, all generated from the same operator registry by changing only the spec.

11 Emergent ant trails

Adding the `sense` operator (a Physarum/Jones three-sensor rule: each ant samples pheromone ahead-left / ahead / ahead-right and turns toward the strongest, while `glide` drives it forward and `deposit` lays pheromone) yields self-reinforcing trails that grow, are followed, and coarsen over long runs (Fig. 5). Listing 3 is the full simulation.

Listing 3: The ant simulation: red cells lay, follow, and reinforce pheromone trails.

```
name: ant
seed: 0
n_frames: 1500
sets:
  cell:
    n: 150
    types: {soft: {fraction: 0.6, youngs: 60}, stiff: {fraction: 0.4, youngs: 300}}
  particle: {parent: cell, per_parent: 60, radius: 0.012}
fields:
  pheromone: {frame: grid, res: 160, couples_to: cell}
operators:
  - {op: glide, at: cell, speed: 300.0, rot: 0.06}              # persistent forward motion
  - {op: sense,  at: "cell[type=soft]", from: pheromone, turn: 0.4, sensor_dist: 0.04, sensor_angle: 0.5}
  - {op: deposit, at: "cell[type=soft]", to: pheromone, rate: 2.0}
  - {op: diffuse, at: pheromone, rate: 0.02}                    # field self-dynamics (was pheromone.diffuse)
  - {op: decay,   at: pheromone, rate: 0.04}
  - {op: mls_mpm_mechanics, at: particle, n_grid: 128, substeps: 8, a_max: 1200, drag: 0.6}
schedule: [aggregate, sense, glide, deposit, diffuse, decay, mls_mpm_mechanics]
```

12 Cells in a maze: foraging and optimization

A harder simulation exercises more of the language: two rigid cell populations start at *home* (bottom-left), navigate a walled *maze* to *food* (top-right), pick food up, and carry it back. It adds

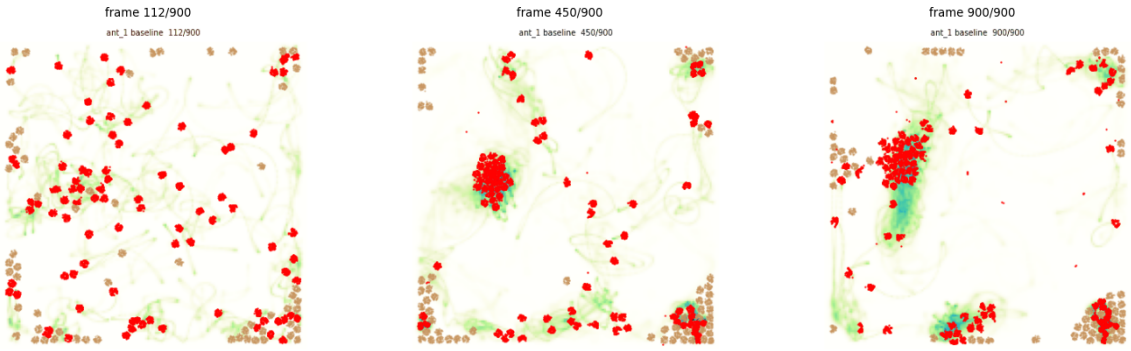


Figure 5: Ant trail-following over a long run (three frames, early to late). A diffuse network of pheromone trails (shaded) forms, reinforces, and *coarsens into a few dominant trails* with ants marching along them (right) — classic stigmergy.

three constructs — `obstacles`, a per-cell `loaded` state, and state-gated selectors (`cell[loaded=0]` seeks food, `cell[loaded=1]` returns home) — and answers the design question *how does one model an obstacle*: a wall is not a pairwise force but a **static occupancy mask reused as a boundary condition** by machinery already present. The same mask (i) zeroes the MPM grid velocity inside walls, so rigid cells cannot enter, and (ii) imposes no-flux on field diffusion, so a chemical *routes around* the walls. Two such fields — one sourced at food, one at home — become navigation gradients that solve the maze; unloaded and loaded cells climb opposite ones.

Because the whole simulation is differentiable and the spec is tiny, the colony can be *optimized*: search the behavioural parameters to deliver food as fast as possible. The objective — units delivered — comes from a discrete pick-up/drop-off and is therefore non-differentiable, so we use a black-box **UCB** search over eight levers (motility speed and rotation, sense gain, MPM drag and force cap, cell stiffness and size, colony size). Over a few dozen simulations it improves delivery roughly five-fold. This echoes the differentiable-simulator design literature, where a *smooth* objective is optimized by gradients through the rollout; the general recipe is **both** — UCB or CEM for the discrete and structural choices, and gradient-through-rollout on a smooth surrogate for the continuous interior.

The optimizer keeps a log of every accepted improvement (the auto-generated `WINNERS.md`), which is itself the finding: it shows *what makes a good forager*. Two parameters move monotonically to their bounds — the colony grows to its cap ($n \approx 120$: more foragers deliver more food) and the cells soften to the floor (youngs $\to 20$: soft, deformable cells squeeze through the maze far better than the rigid ones we started from). Chemotaxis stays strong throughout (gain $\sim$ 340–400) and the cell radius sits at its minimum (smaller cells clear the gaps). The remaining knobs — motility speed and rotation, drag, force cap — are traded off rather than maximized: the best design (`winner_7`) is a balanced, moderate-speed, moderate-drag colony, not any extreme. UCB explores widely (the parameter columns are non-monotonic) while the best-so-far food rises monotonically by construction. The headline is that the search rediscovered, with nothing hand-coded, that a *large swarm of small, soft, strongly-chemotactic cells* forages a maze best.

The difference is visible in the trails themselves (Fig. 7): the initial colony barely establishes a route, whereas the optimized one lays a strong chemoattractant highway that threads both maze

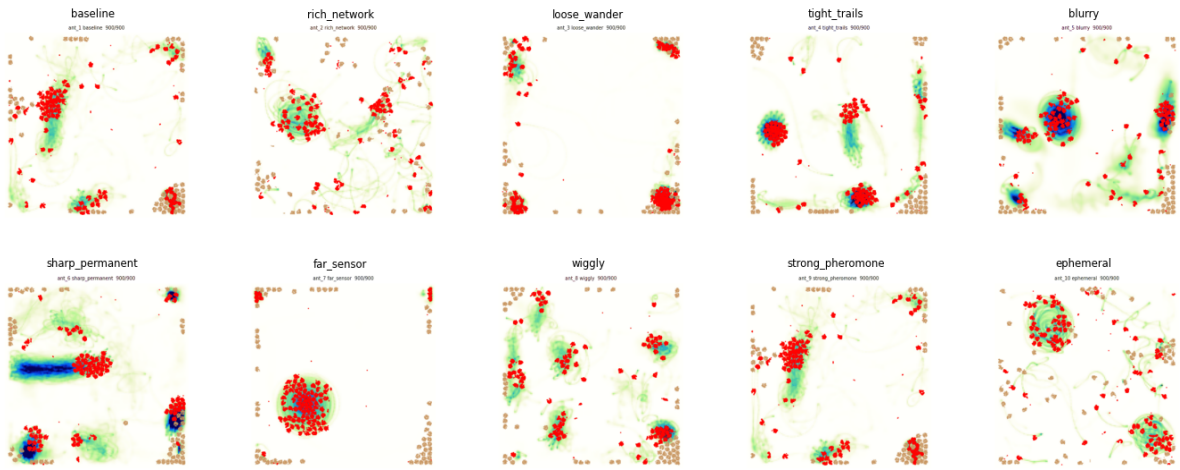


Figure 6: Ten ant variants, each the same `sense+glide+deposit` code with a few parameters changed (turn rate, sensor reach/angle, pheromone diffusion/decay, deposit rate, rotational noise). The parameter axes span the trail-formation regime — from loose wandering to tight reinforced networks.

gaps from home to food.

13 Races: a longitudinal world, an exit boundary, and more levers

The foraging maze generalizes into a family of *races*: a population starts at the left and must reach an exit at the right as fast as possible. Three additions to the language carry them, none touching the engine's generic core.

A longitudinal world. A declarative `world: W` makes the domain the rectangle $[0, W] \times [0, 1]$ on a grid of *square* cells ($n_x = \text{round}(W\, n_y)$); W=4 gives a long left-to-right track. Obstacles may now be **circles** (`[cx,cy,r]`) as well as rectangles, so a porous field of ∼hundreds of round pillars and a real recursive-backtracker maze (genuine corridors and dead ends) are both just obstacle lists, reused as the one MPM/field mask as before.

An exit (efflux) boundary. A cell crossing the finish is handled by an operator: `finish` *freezes* it at the line, while `death` *removes* it (zeroing its particles' mass and parking them off-domain) so it disappears instead of piling up. Both count the crossing in `H.finished` (the race objective) and set a sticky `done` flag; driving operators select `cell[done=0]`, so an exited cell stops being driven or drawn. `death` is the discrete-set counterpart of a field `source`: an open sink, run last in the schedule. Navigation reuses the proven hidden *geodesic* field (no source bands), while only the faint deposited `scent` trail is shown.

More levers, including `boids`. The optimizer (`race_opt.py`, same kNN-UCB as foraging, a winner gif saved whenever the escaped count rises by ten) sweeps the foraging levers *plus the three boids weights* and its radius — `boids.cohesion/align/separate/radius` — twelve in all. This is

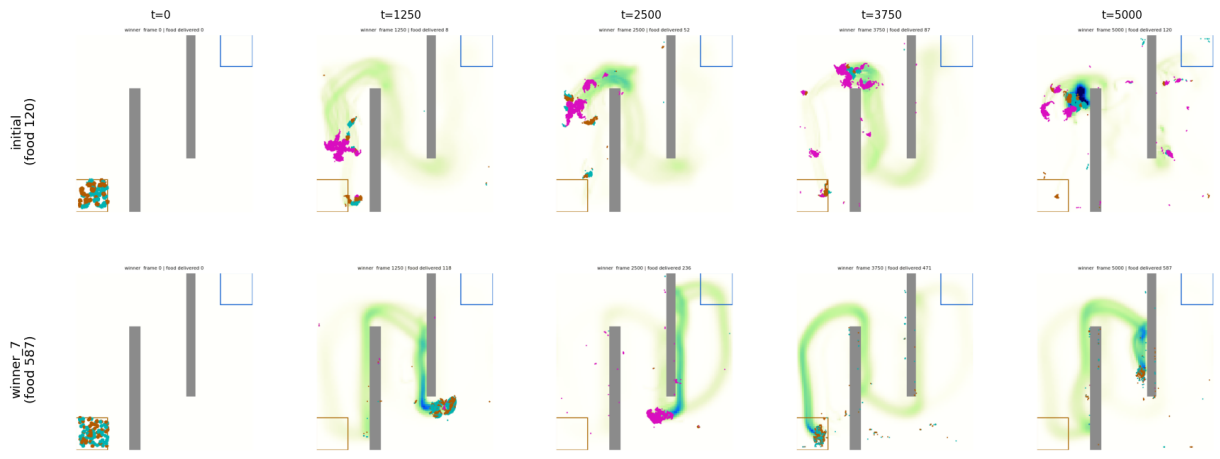


Figure 7: Initial design (top) versus the best found by UCB (bottom, `winner_7`), each a single run shown as five frames from early to late. The optimized colony forms a far more complete and persistent foraging trail.

the point of the operator-parameter view: the search tunes the collective-motion rule (how strongly cells flock and avoid jamming in a tight maze) jointly with body and fleet, not just scalar gains.

14 What the process taught us

1. **The intermediate representation is the whole game.** Ten distinct behaviours and an ant model were all different YAML over the same code.
2. **Selectors are the highest-leverage primitive.** `cell[type=soft]` expresses "only population 1 does X" and lets behaviours coexist; every operator must honour the selector mask.
3. **Verification must check *intent*, not just "it ran".** The hard bugs were physical: unstable diffusion ($dt \cdot D > 0.25$), wrong particle volume, unclamped accelerations blowing up MPM, cell inversion at very low stiffness, GPU non-determinism breaking reproducibility, and — subtlest — the *wrong metric* (net displacement vs. nearest-neighbour clustering).
4. **Honesty about regime.** Self-secreted chemotaxis aggregates only below a density threshold (above it the field is spatially flat — correct physics); the simulation/verify output should state the regime, not silently show the one setting that worked.
5. **Determinism is a feature you must force** (deterministic CUDA kernels), or "reproduce with a new seed" is meaningless.
6. **Obstacles are boundary conditions, not forces.** A wall is one static mask reused by the MPM grid (interior zero-velocity) and the diffusing fields (no-flux); maze navigation then falls out of the field's gradient, with no special pathfinding code.
7. **Cap speed below the CFL limit.** Strong gradients otherwise drive cells to the grid's stability ceiling, where they move ballistically ("asteroids"); an explicit max-speed clamp plus overdamped drag restores smooth, biological motion.

8. **Discrete objectives need black-box search, differentiable ones admit gradients — use both.** Food count (discrete) optimizes well under UCB; a smooth surrogate opens the gradient-through-rollout path, and the two compose: UCB for structure, gradients for the continuous interior.

9. **Reproducibility needs full-precision provenance.** Archiving rounded parameters made a winner re-simulate to a different score; the exact spec (and seed) must be stored, or the run is not bit-reproducible.

Part IV — Building the repository

15 Repository contract

Plexus is not a collection of simulations. It is an interpreter for a formal simulation language. The repository must therefore be built around one rule:

> New biology enters Plexus by adding registered entities, fields, or operators; never by adding simulation-specific logic to the engine.

The engine is the kernel. The spec is the program. The registry is the vocabulary. The validator is the contract gatekeeper. Operators are the only place where new dynamics live.

Every coding change must preserve this separation.

$$\text{spec.yaml} \longrightarrow \text{schema} \longrightarrow \text{registry} \longrightarrow \text{engine} \longrightarrow \text{trajectory}.$$

If a change requires the engine to branch on a simulation name, biological case, operator name, or field name, the abstraction has failed. The correct fix is almost always a new registered operator, a new entity schema, a new field class, or a stronger validator rule.

16 Layering

The repository is divided into strict layers.

- `models/base.py`: the contract layer: `Level`, `Field`, `Hierarchy`, `Operator`, and the semantic operator kinds.
- `models/registry.py`: the only name-to-code map. It registers entities, fields, and operators. The engine must resolve names only through this registry.
- `models/entities.py`: declares state schemas and rendering hints for entity kinds. It is the only source of truth for what columns of `Level.state` mean.
- `operators/*.py`: one operator per file, one concern per operator. Operators declare their `KIND`, `EMIT`, `REQUIRES_PARAMS`, `REQUIRES_TYPE_PROPS`, and optional mechanism-search metadata.
- `fields/*.py`: field state classes. A field stores continuum state and geometry. Field dynamics are operators, not field methods.

- `schema.py`: validates the spec before execution. A spec that passes schema validation should be runnable by the engine.
- `engine.py`: the generic interpreter. It builds the hierarchy, instantiates registered operators, executes the schedule, sums deltas, and integrates each set once per tick.
- `plot.py` or `viz.py`: generic visualization over recorded sets and fields. It must never branch on simulation names.
- `archive.py` and `simulations/`: reproducible gallery and ledger: spec, seed, metrics, thumbnails, mechanism captions, and failure notes.

No layer may reach upward. The engine may import the registry; the schema may import the registry; operators may import base classes and geometry utilities. Operators may not import the engine. The schema may not execute simulations.

17 Operator kinds

Every operator must belong to exactly one kind. The kind is determined by what the operator changes.

Kind	Changes	Returns	Examples
`lateral`	set state within one set	delta	attraction, boids, drag
`aggregate`	parent state from children	delta or assignment	particle-to-cell centroid
`broadcast`	child state from parent	delta	cell force to particles
`exchange`	set–field coupling	delta or field write	deposit, sense, chemotaxis
`field`	field self-state	{}	diffuse, decay, playback, reaction
`rewire`	relations E	{}	radius graph, membrane graph
`structural`	entity membership $\|S\|$	{}	divide, die

This table is normative. Do not reinterpret it locally.

Forbidden shortcuts.

- Do not implement field self-dynamics as `exchange`. Use `kind="field"`.
- Do not implement neighbour rebuilding inside a force operator. Use `rewire`.
- Do not implement division or death by resizing tensors. Use fixed buffers and `occ`.
- Do not implement friction as an integrator option. Use a `drag` operator.
- Do not implement biological behaviour in the engine. Use an operator.

18 The integration invariant

The engine is the only code allowed to integrate position and velocity.

A dynamics operator never writes `pos` or `vel` directly. It returns a delta:

$$\Delta_i = \begin{cases} \text{a velocity} & \text{if EMIT} = \texttt{velocity}, \\ \text{an acceleration} & \text{if EMIT} = \texttt{acceleration}, \\ \text{a substep body acceleration} & \text{if EMIT} = \texttt{mpm\_acceleration}. \end{cases}$$

The engine sums the deltas for a set and integrates once at the end of the tick; `mpm_acceleration` deltas are read instead by the MPM substep (as the external acceleration a_{ext}) and are never engine-integrated. This is one vocabulary, shared by the class attribute `EMIT` and the spec `emit:` override with no translation table; `EMIT` = `None` (a `field` / `rewire` / `structural` operator, or an exchange that writes a field) emits no set delta at all.

Allowed in-place mutations are limited to:

- relations: `edge_index` by `rewire` operators;
- fields: `grid` by `field` or set-to-field exchange operators;
- membership: `occ`, `birth`, `lineage`, and per-node buffers by `structural` operators;
- auxiliary control state: e.g. `heading`, provided it is not part of the engine-integrated state.

Everything else must be returned as a delta.

Engine guard. The engine must check this invariant around every non-mutating operator. The guard must not be disabled globally just because one structural operator exists. Instead, only operators that explicitly declare permission to mutate integrated state may do so. The preferred rule is

```
MAY_MUTATE_INTEGRATED_STATE = False
```

by default. Structural operators that truly need to rewrite state must opt in explicitly and document why.

19 Selector and occupancy rule

Every operator acting on a set receives a selector mask. Every operator must also respect occupancy. The effective mask is always

$$m_i = \mathbf{1}_{\text{selector}}(i)\, w_i, \qquad w_i = \texttt{occ}_i.$$

This applies to all operators, including operators that mutate auxiliary state such as `heading`. Dormant nodes must not move, sense, turn, deposit, attract, repel, divide, die again, or contribute to aggregates. Required pattern:

```
m = selector_mask * lvl.occ
```

For message passing, dormant senders and receivers must both be handled: sender occupancy weights messages; receiver occupancy masks the output.

20 Field rule

A field is pure state: a grid, geometry, and possibly external data. A field does not own its dynamics.

- `deposit`: set → field, `kind="exchange"`.
- `sense` or `chemotax`: field → set, `kind="exchange"`.
- `diffuse`, `decay`, `playback`, `react`: field → field, `kind="field"`.

Retired schedule forms such as `chemical.diffuse` are forbidden. Field updates are ordinary registered operators:

```
operators:
  - {op: diffuse, at: chemical, rate: 0.2}
  - {op: decay,   at: chemical, rate: 0.01}
schedule: [deposit, diffuse, decay, sense]
```

The schema and engine must agree. If the schema accepts a schedule token, the engine must run it. If the engine rejects a token, the schema must reject it first.

21 Aggregate and broadcast rule

Aggregate and Broadcast are not optional conveniences. They are primitive operators of the calculus.

Aggregate. An aggregate operator moves information upward along a parent map, $S_\alpha \to S_\beta$. It must use `Level.parent` and occupancy-weighted reductions, and must not assume a linear scale ladder. Required form:

$$x_\beta(b) = \frac{\sum_{i:\pi(i)=b} w_i \, x_i}{\sum_{i:\pi(i)=b} w_i}.$$

Broadcast. A broadcast operator moves information downward along the same parent map, $S_\beta \to S_\alpha$. It must use `Level.parent` to index parent state for each child, and must respect child occupancy.

Aggregate and Broadcast are implemented as registered operators, not as special string builtins in the engine. The schedule names them like any other operator.

22 Structural rule: divide and die

Structural operators change membership, not tensor shape. All sets are allocated at a fixed buffer size; a node exists when $\texttt{occ}_i > 0$.

Divide. Division activates dormant slots. It must use `Level.spawn` or an equivalent central primitive that copies all per-node buffers safely. A division operator must not hand-copy only `state`; it must preserve all relevant per-node buffers: type, parent, mass, deformation state, lineage, and domain-specific buffers.

Die. Death sets `occ` $\to 0$. If a level has mass, death also sets mass to zero. A dead node should no longer contribute to rewire, lateral interactions, exchange, aggregation, visualization, or metrics.

No resizing. Structural operators never call tensor concatenation to change the number of nodes. They wake or retire fixed slots.

23 Boundary and geometry rule

The world owns geometry.

- Domain width, height convention, boundary type, and obstacles live in `general:`.
- Operators read `H.world_width`, `H.periodic`, and `H.obstacles`.
- Boundary conventions must be shared by all spatial operators.
- A world is $[0, W] \times [0, 1]$ unless the base geometry contract is changed globally.

No operator may define its own incompatible notion of the domain. For finite-difference field gradients, periodic worlds may wrap; wall worlds must not wrap — use boundary-aware padding.

24 State schema rule

Operators must not hard-code state columns unless they are explicitly limited to an entity whose schema is fixed. Preferred form:

```
pos = lvl.get("pos")
vel = lvl.get("vel")
```

The meaning of state columns belongs to `@register_entity`, not to an operator; a new entity kind declares its schema once. Forbidden pattern (except in temporary code with a comment explaining why the entity schema is not yet available):

```
pos = lvl.state[:, :2]
vel = lvl.state[:, 2:4]
```

25 Type and parameter rule

A type is a parametric variant inside one set; a sort is a different set.

- Different state space or different admissible operators: new set.
- Same state space and same operators, different parameter values: type within one set.
- Per-type properties belong in `types:`.
- Operators read per-type properties only through declared `REQUIRES_TYPE_PROPS`.

When assigning type fractions to a buffer, rounding must not silently leave unassigned nodes. The final type receives all remaining nodes.

26 Schema–engine consistency

The schema is the contract gatekeeper; the engine must not discover avoidable errors at runtime. The schema must reject:

- unregistered operators; unknown sets or fields; unknown operator kinds;
- missing `REQUIRES_PARAMS`; missing `REQUIRES_TYPE_PROPS`;
- schedule tokens that do not resolve; retired builtins;
- conflicting prediction orders on the same set; invalid type fractions;
- field operators whose `at:` is not a field; set operators whose `at:` is not a set.

A validated spec should not hit `NotImplementedError` in the engine except for explicitly marked experimental features.

27 Mechanism metadata rule

Every operator should be captionable; this supports the mechanism-search ledger. Operators may declare `MECHANISM_TAGS` and `PARAM_ROLES`. These are not used by the engine — they are used by documentation, the simulation ledger, and agentic mechanism search:

```
MECHANISM_TAGS = ["long_range_attraction", "short_range_repulsion", "coarsening"]
PARAM_ROLES = {"sigma": "interaction_length"}
```

The goal is to make every spec translatable into mechanistic language. (An earlier `MORPHOLOGY_PRIOR` field — a fixed per-operator guess at emergent shape — was removed: morphology is set by the *parameters* of a spec, not by the operator, so the same operator yields a bound cluster, a featureless disc, or a spiral disc depending on its couplings.)

28 Operator naming rule

An operator's name states its *mechanism*, never its *wiring*. Which entities an operator connects is already declared per invocation — `from:` and `to:` name the source and target set or field, and `EMIT` names the state its delta updates — so encoding the endpoints in the name (`agent_to_mpm`, `pulse_to_active_stress`, `field_to_agent`) is redundant and does not scale: every new pair of endpoints would mint a new name (`particle_to_grid`, `grid_to_mesh`, `field_to_particle`, ...). Name the algorithm and let the spec route it:

```
- op: scatter          # mechanism: particle -> grid transfer
  from: cell
  to: mpm_particle
- op: gather           # the same transfer, wired the other way
  from: mpm_particle
  to: cell
```

Thus the transport pair `agent_to_mpm`/`mpm_to_agent` is really `scatter`/`gather` (one particle↔grid transfer, wired by `from:`/`to:`), and `pulse_to_active_stress`/`pulse_to_contraction` are `active_stress`/`active_` — two constitutive laws distinguished by the quantity they emit (a stress tensor $-A\,\hat{n}\hat{n}^{\top}$ vs. a body force), not by their input: the activation source may be calcium, voltage, or a morphogen, so the

name must not hard-code `pulse`. The families `scatter`/`gather`/`deposit`/ `sample`/`aggregate`/`broadcast` are mechanisms; the graph already knows who talks to whom.

> **Naming rule.** Operator names describe mechanisms; `from:` and `to:` describe routing. A directional `X_to_Y` name is admissible only when $X \to Y$ is itself the scientific mechanism, not merely its endpoints.

The test: if two operators differ only in their endpoints and share a mechanism, they are *one* operator wired two ways, not two names — exactly the merge-by-contract rule of §32 read through the routing lens. The name answers only "what does this compute?"; `from:`/`to:`/`EMIT` answer "between what, in which direction."

29 Build loop for a coding agent

A coding agent must follow this loop for every new capability.

1. Identify the category of the requested feature: state, field, relation, entity membership, hierarchy, world, visualization, or validation.
2. Choose the correct home: operator, field, entity schema, validator, engine, or plotter.
3. Before editing code, write the expected spec line.
4. Implement the smallest registered primitive that makes that spec line valid.
5. Add `REQUIRES_PARAMS`, `REQUIRES_TYPE_PROPS`, and mechanism metadata.
6. Add schema validation if the primitive introduces a new contract.
7. Run a minimal spec.
8. Verify three things: the spec validates, the engine runs, and the intended morphology or invariant is measured.
9. Add the run to the gallery with spec, seed, metrics, thumbnail, and mechanistic caption.
10. Do not edit the engine unless the new feature is impossible to express as a registered primitive.

30 Common failure modes and corrections

Failure	Correction
Operator directly updates `pos`	Return a delta; let the engine integrate.
Field self-update registered as exchange	Use `FieldUpdate`, `kind="field"`.
Dormant nodes still act	Multiply the selector mask by `occ`.
Retired builtin accepted by schema	Remove it from schema and engine; use a registered operator.
Rewire hidden inside a force law	Split into `radius_graph` + a lateral operator.
Structural op resizes tensors	Use a fixed buffer + `occ`.
One structural op disables all guards	Guard per operator, not globally.
Operator hard-codes `state[:, :2]`	Use `lvl.get("pos")` and entity schemas.
Type fractions leave leftovers	Assign the remainder to the final type.
Engine branches on a simulation name	Move the behaviour into a registered operator.
Plotter branches on a simulation name	Render generically from the data layout and render hints.
Metric says the run succeeded but the morphology is wrong	Add intent-level verification.

31 Definition of done

A new operator or capability is complete only when all of the following are true:

- It is registered. Its kind is correct. Its capability contract is declared.
- It respects selector masks and occupancy.
- It does not mutate integrated state unless explicitly structural.
- It has a minimal spec. The spec validates. The engine runs without special cases.
- The output has an intent metric.
- The result is archived with spec, seed, metrics, and caption.

If any item is missing, the feature is not part of Plexus yet — it is only prototype code.

32 From prototypes to the Plexus core

Plexus is developed through scientific prototypes: embryo, cardio, active matter, and other domain-specific campaigns routinely introduce operators before their final abstraction is known. A prototype clones an existing model — a vertex tissue, a reaction–diffusion patterner, a self-propelled-Voronoi sheet — reproduces it as a throwaway operator set, and explores it freely. Code is therefore *not* moved into the core merely because it works in one prototype. Moving code from a prototype into Plexus is not copying useful files; it is **promoting a mechanism into a stable operator language**. Promotion happens only after the mechanism's family, contract, naming, dimensional assumptions, and tests have stabilized. The rule is simple: *prototype freely, promote conservatively.*

Promotion passes through five gates.

1. **Promote families, not isolated operators.** Before a mechanism is moved, it is assigned to a family — motion/migration, chemical signalling, field sampling, continuum (MPM) mechanics, growth, activation, coupling, measurement/scorecard, or visualization. An operator enters the core only when its family is clear. This keeps the registry a small set of reusable physical primitives rather than a flat pile of one-off biological events.

2. **Merge by contract, not by surface similarity.** Two prototype operators collapse into one canonical operator with explicit options only when they share the same contract and differ solely in routing, source field, or mode — for example `chemotaxis`+`chemo_force`→`chemotax`, `mpm_drag`+`drag`→`drag` with an `emit` option, and `pulse_stimulus`+`phase_delay_pulse`→`activation_pulse`. When the contract differs they stay separate and share only backend utilities: `active_force` emits an `mpm_acceleration`, whereas `active_stress` writes an active stress and emits nothing — those must not become one muddy mode-dependent operator. Merging is thus governed by physical and computational semantics, never by superficial code similarity.

3. **Stabilize names before promotion.** Every promoted operator receives one canonical name and one canonical vocabulary: its Python class, registry key, YAML name, emitted prediction type, state fields, schema path, scorecard metric names, and the terminology in this manuscript must all agree. The `EMIT` refactor is the model — `velocity`, `acceleration`, `mpm_acceleration`, with no synonyms, no legacy aliases, and no `first_derivative`/`second_derivative` drift. Legacy synonyms are removed *at* promotion time rather than preserved indefinitely, so that Plexus stays a language and not a collection of historical aliases.

4. **Require 2D → 3D design at promotion time.** The 3D implementation need not exist immediately, but the interface must make 3D possible. Each promoted operator declares `SUPPORTED_DIMS` ([2] or $[2, 3]$) and records which quantities are scalars, vectors, and tensors, which assumptions are 2D-only, and what the 3D generalization would be. A planar angle θ or a signed chirality is 2D-specific and must state its axis/vector/handedness generalization; `Field.sample` and `Field.grad_at` are naturally dimension-generic. The goal is to prevent 2D geometry from silently entering the core abstraction.

5. **Promotion requires tests and paper reconciliation.** A promotion is complete only when code, specifications, tests, and documentation move together. Behaviour-preserving migrations require a byte-identical trajectory check against the originating prototype on a representative spec; new mechanisms require a no-op/ablation test, schema validation, and at least one minimal live spec. The operator catalog and this manuscript are then regenerated or reconciled so the paper describes the live registry rather than an older prototype.

6. **Cite the source paper and repository.** Because a prototype begins by cloning a published model, the promoted operator must carry that provenance: the originating paper and code repository are recorded in the operator's docstring and in this manuscript, so a reader can trace each core primitive back to the science it was built from. Promotion inherits credit, it does not erase it.

As a worked example, the inverse-square `squared_law` operator is a single such promotion: it unifies the electrostatic Coulomb law (ported from ParticleGraph's `PDE_E`) with softened Newtonian gravity (from Philip Mocz, *Create Your Own N-body Simulation (With Python)*, 2020, vendored under `papers/nbody-python/`) into one `law`-switched operator. Both are the same contract — an

add-aggregated pairwise $1/r^2$ force emitted as an acceleration — differing only in the source charge (`charge` vs. `mass`), the sign/receiver convention (like-repel vs. attract), and the summation range (neighbour graph vs. all-pairs). The Coulomb path is byte-identical to the pre-merge operator; the gravity path reproduces the prototype's force exactly. The galaxy prototype is thereby retired into the core as a spec, not a fork.

This promotion discipline is the same reconvergence process the repository was built for: sibling prototype frameworks that had drifted apart are pulled back into one shared vocabulary, one operator at a time. Prototypes are where mechanisms are discovered; the core is where they become a language.

33 The principle

Plexus grows by expanding its vocabulary, not by weakening its grammar.

A coding agent should therefore prefer adding one small, typed, registered primitive over adding a clever shortcut. Shortcuts make one simulation work. Primitives make the language larger.

Appendix

A Plexus and Lean

At first glance Plexus and a proof assistant such as Lean appear unrelated. One simulates living systems; the other verifies mathematics. Yet they share a surprisingly similar architecture. Both seek to construct complex objects from a small set of primitive building blocks while preserving correctness. Understanding this correspondence helps clarify what Plexus actually is: not merely a simulator, but a formal system for describing biological models.

A.1 A brief introduction to Lean

Lean is an interactive theorem prover developed to formalize mathematics. A user states a theorem and constructs a proof. Unlike a traditional mathematical proof written for humans, a Lean proof must be expressed in a precise formal language that the Lean kernel can verify mechanically.

For example, the mathematical statement

$$a + b = b + a$$

can be expressed as a theorem in Lean:

```
theorem add_comm (a b : Nat) : a + b = b + a := ...
```

The dots represent a proof. Once supplied, Lean checks every step and certifies that the conclusion follows from the rules of the system.

Lean is built around several layers.

1. **Types**. Every object belongs to a type: natural numbers, real numbers, sets, functions, and so on.

2. **Terms**. Concrete objects inhabit types. The number 3 is a term of type `Nat`. A function is a term of a function type.
3. **Theorems**. A theorem is a proposition that one wishes to prove.
4. **Proof terms**. A proof is itself an object. Under the Curry–Howard correspondence, propositions behave like types and proofs behave like terms inhabiting those types.
5. **The kernel**. A small trusted program checks that a proof is valid.
6. **Tactics**. Convenient automation that helps construct proofs. Tactics can be sophisticated and even unreliable internally because the kernel checks the result afterward.
7. **Mathlib**. A large library of previously formalized mathematics.

The key idea is separation of concerns. The user writes proofs, tactics help construct them, and the kernel verifies them. Trust resides in the kernel rather than in the process that produced the proof.

A.2 The structural correspondence

Plexus exhibits a remarkably similar decomposition. The correspondence is not perfect, but it is useful.

Lean	**Plexus**	**Role**
Types	Sets S_α and Fields F	primitive objects
Terms	Entities (cells, particles, metabolites)	instances of those objects
Functions	Operators	transformations
Proof composition	Schedule	composition of transformations
Type checker	Schema validator	correctness gate
Tactics	LLM compiler	search / generation layer
Theorem	Intent	desired outcome
Proof term	Simulation specification	executable description
Kernel	Engine	trusted interpreter
Mathlib	Operator registry	reusable library of primitives

Several rows deserve explanation.

A set in Plexus plays a role similar to a type in Lean. Just as Lean distinguishes natural numbers from matrices or groups, Plexus distinguishes cells, particles, metabolites, nuclei, and fields. Operators declare which kinds of objects they act upon, exactly as functions declare their input types.

Entities are analogous to terms. A particular cell is an element of the set `cell`, just as the number 3 is an element of the type `Nat`.

The operator registry resembles a mathematical library. New functionality is added by registering a new operator rather than modifying the engine itself. Existing simulations continue to work because the vocabulary only grows.

The strongest correspondence is architectural. The simulation specification (`spec.yaml`) is generated by a human or by an LLM. It is not trusted. The schema validator checks that the specification is well formed before execution. Only after passing validation is it handed to the engine. This mirrors the relationship between tactics and the Lean kernel.

A.3 Why the analogy is imperfect

Despite these similarities, Plexus is not a theorem prover.

Lean is fundamentally concerned with *truth*. Given a proposition P, Lean determines whether a proof exists. If the kernel accepts the proof, the theorem is established with mathematical certainty.

Plexus is concerned with *dynamics*. Given an initial state and a collection of operators, Plexus computes

$$x_{t+1} = \Phi(x_t).$$

The result is not a proof but a trajectory.

This difference has important consequences.

First, the Plexus validator checks only *well-formedness*, not correctness. A specification that passes validation may still represent a poor biological model. The analogue of theorem proving in Plexus is not validation but experimental verification.

Second, Lean computations terminate. A proof either checks or it does not. Plexus simulations may be unstable, chaotic, or sensitive to parameters. The engine integrates a dynamical system rather than reducing an expression to normal form.

Third, Plexus is differentiable. Gradients flow through operators, schedules, and rollouts. This allows optimization of model parameters and even biological designs. There is no direct analogue of differentiation in theorem proving.

A.4 What Plexus borrows from theorem provers

The value of the comparison is therefore not that Plexus proves theorems. Rather, it inherits a design philosophy from systems such as Lean.

The first lesson is to separate a symbolic description from its execution. A simulation should be represented explicitly rather than being scattered across code.

The second lesson is to separate a trusted kernel from untrusted generation. The LLM may produce a simulation specification, but correctness of the specification is established by the validator and the engine, not by the model that wrote it.

The third lesson is that a small set of primitives can support a large space of constructions. Lean builds mathematics from types, functions, and proof rules. Plexus builds biological models from sets, fields, operators, and schedules.

A.5 A language and calculus of living systems

Seen from this perspective, Plexus is best understood as a formal language and calculus for living systems.

The language is the simulation specification. It provides a symbolic vocabulary for describing entities, fields, relations, and schedules.

The calculus is the collection of operators acting on those objects. Lateral, Aggregate, Broadcast, Exchange, Rewire, Divide, and Die are the primitive transformations from which larger biological mechanisms are assembled.

The engine plays the role of an interpreter, turning symbolic descriptions into dynamical trajectories.

This is ultimately the deepest connection to Lean. Both systems seek a compact set of primitives from which a much larger space of structures can be constructed. Lean does this for mathematics. Plexus attempts to do it for biological systems.

B Plexus, World Models, and Mechanism Search

The goal of Plexus is not merely to simulate biological systems. It is to provide a language in which biological mechanisms can be expressed, compared, and discovered.

This distinction became apparent during an agentic modeling experiment on *Dictyostelium* aggregation. An agent was tasked with reproducing a real biological morphology from experimental observations. The search successfully explored many parameterizations of chemotaxis, adhesion, relay signaling, saturated sensing, density repulsion, and deformable mechanics. Several mechanisms were falsified, and one family of models transiently reproduced the target morphology before collapsing at longer times.

At first sight this appeared to be a failure of optimization. In retrospect it revealed a more important limitation: the search explored parameters within a mechanism, but not the space of mechanisms themselves.

Human observers immediately proposed alternative explanations:

- long-range attraction between aggregates,
- external radial fields,
- soft-body fusion,
- coarsening dynamics,
- three-dimensional aggregation projected into two dimensions.

None of these hypotheses were explored. The reason was not that they were unlikely, but that they were absent from the search space. The loop performed local refinement around an existing model rather than proposing qualitatively different explanatory worlds.

The lesson is that scientific discovery requires two distinct searches:

$$\text{mechanism search} \quad \gg \quad \text{parameter search}.$$

The first determines *what kind of world* might explain an observation. The second determines *which parameters* within that world are most consistent with the data.

C World Models and Plexus

Modern world models approach this problem from the opposite direction.

A predictive model such as JEPA or a video foundation model learns a latent representation

$$\text{video} \longrightarrow z_t \longrightarrow z_{t+1},$$

where the latent state z_t is optimized for prediction. Such models can be extremely successful at forecasting future observations, but the latent representation is not required to correspond to interpretable biological mechanisms.

Plexus instead begins from explicit objects:

$$(S_\bullet, F_\bullet, \pi_\bullet)$$

and explicit operators acting on them. Cells, particles, fields, and interactions remain visible throughout the computation.

The difference is therefore not prediction versus simulation. Both can predict.

The difference is the object being searched.

	World Models	**Plexus**
Primitive representation	latent variables	sets, fields, operators
Search space	latent trajectories	mechanisms and parameters
Primary strength	prediction	hypothesis testing
Interpretability	low–moderate	high
Counterfactuals	implicit	explicit
Scientific hypotheses	difficult to extract	native objects

The two approaches should therefore be viewed as complementary rather than competitive. A world model discovers regularities. Plexus evaluates candidate mechanisms capable of generating those regularities.

D A Mechanistic World Model

The Dicty experiment suggests that a successful discovery system requires an intermediate layer between observations and simulations.

Rather than learning latent dynamics directly, the objective is to learn a mapping

$$\text{simulation specification} \longrightarrow \text{mechanistic description}.$$

Every simulation in the Plexus gallery already contains three linked objects:

$$(\text{specification},\ \text{trajectory},\ \text{mechanistic explanation}).$$

For example:

> "Long-range attraction produces coarsening and eventual fusion into a dominant aggregate."

or

> "A reaction–diffusion field generates filamentary transport networks through local activation and long-range inhibition."

These descriptions form a growing knowledge base connecting operators and morphologies.

The objective is therefore not to learn

$$\text{video} \rightarrow \text{future video},$$

but rather

$$\text{video} \rightarrow \text{mechanistic hypotheses.}$$

This representation remains interpretable because its primitives are the same objects used by the simulator itself.

E The Mechanism Search Tree

Once mechanisms are represented explicitly, the search process changes.

Traditional optimization explores a parameter space:

$$\theta_1, \theta_2, \ldots$$

for a fixed model.

Plexus instead organizes discovery as a hierarchical search over mechanisms.

$$\text{Observation} \rightarrow \text{Mechanism Family} \rightarrow \text{Mechanism Composition} \rightarrow \text{Parameter Refinement.}$$

The resulting search tree resembles

```
Explain observed morphology
|
+-- Long-range attraction
|   +-- attraction radius
|   +-- attraction strength
|
+-- Radial field
|   +-- field shape
|   +-- sensing gain
|
+-- Soft-body fusion
|   +-- Young's modulus
|   +-- surface tension
|
+-- Chemotactic relay
    +-- diffusion
    +-- decay
```

A bandit strategy such as UCB can then allocate computation not only between parameter values but also between competing explanatory worlds.

The crucial distinction is that branches correspond to scientific hypotheses, not merely numerical settings.

F Roadmap

The proposed roadmap consists of four stages.

Stage 1: Mechanistic Captioning. Construct a library of simulations spanning the operator vocabulary of Plexus. Each simulation is paired with a structured mechanistic description.

$$\text{spec} \leftrightarrow \text{mechanism} \leftrightarrow \text{video}.$$

Stage 2: Mechanistic Retrieval. Given a new observation, retrieve simulations with similar morphology and collect their associated explanations.

Stage 3: Mechanism Search. Use the retrieved explanations to initialize a mechanism search tree. The agent explores families, compositions, and parameterizations using simulation-guided evaluation.

Stage 4: Mechanistic World Models. Train models that map observations directly to distributions over candidate mechanisms. The world model no longer predicts future pixels. Instead, it predicts which explanations are worth testing.

$$\text{Observation} \to \text{Mechanism Prior} \to \text{Plexus Search} \to \text{Simulation} \to \text{Falsification}.$$

In this view, Plexus is neither a simulator nor a world model alone. It is a language in which candidate worlds can be written, searched, and falsified.

Construct	Semantics	Example
`general:` — the run and the world (the space + the clock)		
`name`	run identifier	`name: aggregate`
`seed`	RNG seed — fixes determinism	`seed: 0`
`n_frames`, `dt`	ticks to run; the time step	`n_frames: 250`
`boundary`	the space: `periodic` (torus) or `wall` (clamp)	`boundary: periodic`
`world`	domain width W; world is $[0, W]\times[0, 1]$ ($W>1$ longitudinal)	`world: 4.0`
`obstacles`	static walls (rects or `[cx,cy,r]` circles); a BC for the MPM grid + field diffusion	`[[.3,0,.36,.7]]`
Set (S_k — an entity level / ECS entity)		
`n`	size of a top-level set	`n: 100`
`parent`	the set that contains this one — a containment edge $\pi_{A\to B}$ (many sets may share a parent)	`parent: cell`
`per_parent`	children per parent	`per_parent: 100`
`role`	this set's role inside its parent (membrane / cytoplasm / nucleus...)	`role: membrane`
`types`	named sub-populations: fraction + per-type properties	`{soft: {fraction: .5, youngs: 12}}`
Field (a continuum)		
`frame`	discretization (a grid)	`frame: grid`
`res`	grid resolution	`res: 96`
`diffusion`, `decay`	PDE coefficients	`diffusion: 0.1`
`couples_to`	the one set this field exchanges with	`couples_to: cell`
`source`	fixed source region (imposes a gradient)	`source: [.82,.82,1,1]`
`init`	uniform initial fill (self-generated runs)	`init: 1.0`
`navigation`	`geodesic` (precomputed external gradient) or `dynamic` (self-generated)	`navigation: geodesic`
Operator (a system; one per list entry)		
`op`	which registered operator to run	`op: sense`
`at`	selector — the subset it acts on	`at: cell[type=soft]`
`to`/`from`	field an Exchange writes to / reads from	`from: chemical`
⟨params⟩	operator-specific arguments	`gain: 150`
Selector (sub-grammar)		
`set`	every node of the set	`cell`
`set[attr=val]`	nodes matching `attr` (re-checked each tick)	`cell[loaded=1]`
Schedule (sub-grammar; per-tick order)		
operator name	run that operator this tick	`sense`
`[a, b]`	a group run in the same stage	`[glide, sense]`
`aggregate`	a registered op: Aggregate↑ (parent = mean of children)	`aggregate`
`diffuse`	a field op: advance that field one step (Laplacian)	`diffuse`
`plotting:` (render style only — never affects dynamics)		
`colormap`, `point_size`, `background`	how the sets are drawn	`background: black`

Table 2: The simulation language. Each block has its own home in the spec: `general:` is the world and clock, `sets:` are the entities, `operators:` the dynamics, `schedule:` the composition, `plotting:` the style. Integration is implicit (each set is integrated once at the end of a tick), so there is no `integrate` token.